TD 10: Inférence de type

1 Types monomorphes

Voici la syntaxe du langage monomorphe de ce TD.

TYPE

$\tau ::=$	α	(Variable de types)
	num	(type des nombres)
	str	(type des chaînes)
	$\tau_1 \to \tau_2$	(type des fonctions)

EXPRESSION

$e ::=$	x	(variable)
	n	(nombre littéral)
	s	(chaîne littérale)
	$e_1 + e_2$	(addition)
	$e_1 \,\hat{}\, e_2$	(concaténation)
	`len`(e)	(longueur)
	$e_1\ e_2$	(application)
	`fun` $x \Rightarrow e$	(abstraction)
	`let` $x = e_1$ `in` e_2	(définition)

Priorité :

(définition) < (abstraction)
< (concaténation)
< (addition)
< (application)

Associativité à gauche :
addition,application, concaténation

Associativité à droite :
type des fonctions

Lieur :

définition : x dans e_2
abstraction : x dans e

On rappelle les règles de génération de contraintes $\Gamma \vdash e : \tau \triangleright \mathscr{P}$ vues en CM.

VAR
$$\frac{}{\Gamma, x : \tau \vdash x : \tau \triangleright \emptyset}$$

NUM
$$\frac{}{\Gamma \vdash n : \text{num} \triangleright \emptyset}$$

ADD
$$\frac{\Gamma \vdash e_1 : \tau_1 \triangleright \mathscr{P}_1 \qquad \Gamma \vdash e_2 : \tau_2 \triangleright \mathscr{P}_2}{\Gamma \vdash e_1 + e_2 : \text{num} \triangleright \mathscr{P}_1 \cup \mathscr{P}_2 \cup \{\tau_1 \stackrel{?}{=} \text{num}, \tau_2 \stackrel{?}{=} \text{num}\}}$$

FUN
$$\frac{\Gamma, x : \alpha \vdash e : \tau \triangleright \mathscr{P} \qquad \alpha \text{ fraîche}}{\Gamma \vdash \texttt{fun}\ x \Rightarrow e : \alpha \to \tau \triangleright \mathscr{P}}$$

APP
$$\frac{\Gamma \vdash e_1 : \tau_1 \triangleright \mathscr{P}_1 \qquad \Gamma \vdash e_2 : \tau_2 \triangleright \mathscr{P}_2 \qquad \alpha \text{ fraîche}}{\Gamma \vdash e_1\ e_2 : \alpha \triangleright \mathscr{P}_1 \cup \mathscr{P}_2 \cup \{\tau_1 \stackrel{?}{=} \tau_2 \to \alpha\}}$$

La résolution de contraintes $\mathscr{P}, \mathscr{S} \Rightarrow \mathscr{P}', \mathscr{S}'$ ou $\mathscr{P}, \mathscr{S} \Rightarrow$ `echec` a été définie de la manière suivante en CM.

- Trivial : $\{\tau \stackrel{?}{=} \tau\} \cup \mathscr{P}, \mathscr{S} \Rightarrow \mathscr{P}, \mathscr{S}$
- Décomposition : $\{\tau_1 \to \tau_2 \stackrel{?}{=} \tau_3 \to \tau_4\} \cup \mathscr{P}, \mathscr{S} \Rightarrow \{\tau_1 \stackrel{?}{=} \tau_3, \tau_2 \stackrel{?}{=} \tau_4\} \cup \mathscr{P}, \mathscr{S}$
- Conflit : $\{\tau_1 \to \tau_2 \stackrel{?}{=} \text{num}\} \cup \mathscr{P}, \mathscr{S} \Rightarrow$ `echec`
- Élimination de variable : si $\alpha \notin \text{fv}(\tau)$, alors $\{\alpha \stackrel{?}{=} \tau\} \cup \mathscr{P}, \mathscr{S} \Rightarrow \mathscr{P}[\alpha \leftarrow \tau], \mathscr{S}[\alpha \leftarrow \tau] \cup \{\alpha \mapsto \tau\}$
- Circularité : si $\alpha \in \text{fv}(\tau)$ et $\alpha \neq \tau$, alors $\{\alpha \stackrel{?}{=} \tau\} \cup \mathscr{P}, \mathscr{S} \Rightarrow$ `echec`

Question 1 – Écrire les règles de génération de contraintes pour le `let` et la longueur.

Question 2 – Comment prendre en compte le type des chaînes dans la résolution de contraintes ?

Question 3 – Donner un argument informel justifiant pourquoi la résolution de contraintes termine.

Question 4 – Appliquer les génération et résolution de contraintes aux expressions suivantes :

1. `fun` $f \Rightarrow f\,(f\,2)$
2. $g\,(f\,2)\,(f$ `"coucou"`$)$
3. `fun` $f \Rightarrow f\,f$

Ce calcul a la propriété du typage principal : chaque expression admet un typage Γ, τ tel que tout autre typage de cette expression peut être obtenu de Γ, τ avec une substitution.

Question 5 – Formaliser la propriété du typage principal telle qu'énoncée ci-dessus.

Question 6 – Quelle est la différence entre la propriété du typage principal et celle du type résultat principal ?

2 Types polymorphes

On rappelle la syntaxe du noyau ML polymorphe.

SCHÉMA DE TYPE

$$\sigma ::= \tau \quad \text{(type)} \qquad \forall\alpha.\sigma \quad \text{(schéma de type)}$$

TYPE

$$\tau ::= \alpha \quad \text{(variable de type)} \qquad \tau_1 \to \tau_2 \quad \text{(type des fonctions)}$$

EXPRESSION

$$e ::= x \quad \text{(variable)} \qquad e_1\ e_2 \quad \text{(application)} \qquad \texttt{fun}\ x \Rightarrow e \quad \text{(abstraction)} \qquad \texttt{let}\ x = e_1\ \texttt{in}\ e_2 \quad \text{(définition)}$$

Priorités :

(définition) < (abstraction) < (application)

Associativité à gauche :
application

Associativité à droite :
type des fonctions

Lieurs :
définition : x dans e_2
abstraction : x dans e

Étant donné 2 substitutions $\mathscr{S}_1$ et $\mathscr{S}_2$, la composition $\mathscr{S}_2 \circ \mathscr{S}_1$ est définie telle que pour tout α, $(\mathscr{S}_2 \circ \mathscr{S}_1)(\alpha) = \mathscr{S}_2(\mathscr{S}_1(\alpha))$. Étant donné Γ (sans variable de type libre) et e, l'algorithme $W(\Gamma \vdash e) = (\mathscr{S}, \tau)$ (en Figure 1) calcule $\mathscr{S}, \tau$ tels que $\Gamma \vdash e : \mathscr{S}(\tau)$ et $\mathscr{S}(\tau)$ est le type principal de Γ, e.

Question 7 – Appliquer l'algorithme W avec $\vdash$ `let` f `=` `fun` $x \Rightarrow x$ `in` $f\,f$.

VAR
$$\frac{\sigma = \forall\alpha_1 \ldots \alpha_n.\tau \qquad \beta_1 \ldots \beta_n \text{ fraîches}}{W(\Gamma, x : \sigma \vdash e) = (\emptyset, \tau\{\beta_i / \alpha_i\})}$$

FUN
$$\frac{W(\Gamma, x : \alpha \vdash e) = (\mathscr{S}, \tau) \qquad \alpha \text{ fraîche}}{W(\Gamma \vdash \texttt{fun}\ x \Rightarrow e) = (\mathscr{S}, \mathscr{S}(\alpha) \to \tau)}$$

APP
$$\frac{W(\Gamma \vdash e_1) = (\mathscr{S}_1, \tau_1) \qquad W(\mathscr{S}_1(\Gamma) \vdash e_2) = (\mathscr{S}_2, \tau_2) \qquad \{\mathscr{S}_2(\tau_1) \stackrel{?}{=} \tau_2 \to \alpha\} \Rightarrow^* \mathscr{U} \qquad \alpha \text{ fraîche}}{W(\Gamma \vdash e_1\ e_2) = (\mathscr{U} \circ \mathscr{S}_2 \circ \mathscr{S}_1, \mathscr{U}(\alpha))}$$

LET
$$\frac{W(\Gamma \vdash e_1) = (\mathscr{S}_1, \tau_1) \qquad W(\mathscr{S}_1(\Gamma), x : \mathsf{gen}_{\mathscr{S}_1(\Gamma)}(\tau_1) \vdash e_2) = (\mathscr{S}_2, \tau_2)}{W(\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2) = (\mathscr{S}_2 \circ \mathscr{S}_1, \tau_2)}$$

FIGURE 1 – Algorithme W